

DWS Project D2 – Design Memo

1. Table Inventory

- **Member** – Stores member information. Primary key: member_id.
- **Circle** – Stores circle information and its fixed contribution amount. Primary key: circle_id.
- **Membership** – Represents the many-to-many relationship between members and circles. Composite primary key: (member_id, circle_id).
- **Cycle** – Stores the cycles belonging to each circle. Primary key: cycle_id.
- **Contribution** – Records a member's contribution for a specific cycle. Primary key: contribution_id.
- **Bid** – Stores bids submitted by members for each cycle. Primary key: bid_id.
- **Payout** – Records the member receiving the payout for a cycle. Primary key: cycle_id.
- **DiscountShare** – Stores the discount amount redistributed to members after a payout. Primary key: share_id.

2. Constraint Decisions

- One payout per cycle: **cycle_id** is used as the PRIMARY KEY of Payout and also as a foreign key referencing **Cycle(cycle_id)**. Since a primary key is unique, a cycle cannot have more than one payout.
- Fixed contribution amount: **contribution_amount** is stored once in Circle and defined as **NUMERIC(10,2) NOT NULL CHECK (contribution_amount > 0)**. This keeps one fixed contribution amount for the circle without repeating it in every contribution row.
- **UNIQUE (member_id, cycle_id)** in Contribution prevents duplicate contributions by the same member in the same cycle.
- **UNIQUE (member_id, cycle_id)** in Bid prevents a member from submitting more than one bid for the same cycle.

3. Mapping Notes

- The many-to-many relationship between Member and Circle was mapped to the Membership table because a many-to-many relationship cannot be represented directly by placing a single foreign key in either table. Membership therefore uses member_id and circle_id as foreign keys and as a composite primary key.
- The one-to-many relationship between Circle and Cycle was mapped by placing circle_id as a foreign key in Cycle because each cycle belongs to one circle, while one circle can have many cycles.
- The relationships from Member and Cycle to Contribution and Bid were represented using member_id and cycle_id as foreign keys because each contribution and bid must be associated with a specific member and a specific cycle.
- Cycle and Payout were kept as separate tables because a cycle may exist without a payout while it is still open, but a payout cannot exist without a cycle. cycle_id is used as both the primary key and foreign key in Payout to enforce a maximum of one payout per cycle.
- The relationships involving DiscountShare were represented using payout_cycle_id and member_id as foreign keys because each discount share belongs to one payout and is received by one member.

4. Known Limitations

- The schema does not automatically select the highest bid as the winning bid; this is currently handled through queries or application logic.
- Discount shares are stored in DiscountShare, but their amounts are not automatically calculated by the database.
- More complex rules, such as preventing a member from receiving another payout before all eligible members have received one, would require additional transaction or database logic in a later stage.
- The fixed contribution amount is stored at the Circle level, so the current design does not support different contribution amounts for individual members or cycles.